ASSIGNMENT-12.3

R.SATHWIK

2303A52T01

Q: Lab 12: Algorithms with AI Assistance Sorting, Searching, and Algorithm Optimization Using AI Tools

Lab Objectives

The objectives of this laboratory exercise are to:

Week3 - Wednesday

- Apply AI-assisted programming techniques to implement sorting and searching algorithms.
- Analyze and compare algorithm efficiency using time and space complexity.
- Understand how AI tools can suggest optimizations and alternative algorithmic approaches.
- Strengthen problem-solving skills through real-world, data-driven scenarios.

Learning Outcomes

After completing this lab, students will be able to:

- Implement and optimize classic algorithms using AI-assisted coding tools.
- Compare multiple algorithms for the same problem and justify their selection.
- Measure and analyze runtime performance using experimental data.
- Critically review and refine AI-generated algorithmic solutions.

Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.

Tasks

1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:

o Quick Sort o

Merge Sort

3. Measure and compare runtime performance for large datasets.
4. Write a function to display the top 10 students based on CGPA.

Expected Outcome

• Correctly sorted student records.

• Performance comparison between Quick Sort and Merge Sort.

• Clear output of top-performing students.

Task 2: Implementing Bubble Sort with AI Comments •

Task: Write a Python implementation of Bubble Sort.

• Instructions:

• Students implement Bubble Sort normally.

• Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).

• Request AI to provide time complexity analysis.

• Expected Output:

• A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.

Task 3: Quick Sort and Merge Sort Comparison

• Task: Implement Quick Sort and Merge Sort using recursion.

• Instructions:

• Provide AI with partially completed functions for recursion.

• Ask AI to complete the missing logic and add docstrings.

• Compare both algorithms on random, sorted, and reverse-sorted lists.

• Expected Output:

• Working Quick Sort and Merge Sort implementations.

• AI-generated explanation of average, best, and worst-case complexities.

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Task:

• Use AI to suggest the most efficient search and sort algorithms for this use case.

• Implement the recommended algorithms in Python.

• Justify the choice based on dataset size, update frequency, and performance requirements.

Expected Output:

• A table mapping operation → recommended algorithm → justification.

• Working Python functions for searching and sorting the inventory.

Task 5: Real-Time Stock Data Sorting & Searching Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

• Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).

• Implement sorting algorithms to rank stocks by percentage change.

• Implement a search function that retrieves stock data instantly when a stock symbol is entered.

• Optimize sorting with Heap Sort and searching with Hash Maps.

• Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

CODE: import random import time

from typing import List, Dict, Any

# ----------

# Task 1: Sorting Student Records

# ----------

class Student: def __init__(self, name: str, roll:

str, cgpa: float):

self.name = name

self.roll = roll

self.cgpa = cgpa

def __repr__(self):

return f"{self.name} ({self.roll}) - CGPA: {self.cgpa}"

def quick_sort_students(arr: List[Student]) -> List[Student]: """Sorts

students by CGPA in descending order using Quick Sort."""  if len(arr) <=

1: return arr  pivot = arr[len(arr) // 2].cgpa  left = [x for x in arr if

x.cgpa > pivot]  middle = [x for x in arr if x.cgpa == pivot]  right = [x for x

in arr if x.cgpa < pivot]  return quick_sort_students(left) + middle +

quick_sort_students(right)

```
def merge_sort_students(arr: List[Student]) -> List[Student]:     """Sorts
students by CGPA in descending order using Merge Sort."""     if len(arr)
<= 1:        return arr     mid = len(arr) // 2     left =
merge_sort_students(arr[:mid])     right =
merge_sort_students(arr[mid:])     return merge_students(left, right)

def merge_students(left: List[Student], right: List[Student]) -> List[Student]:
result = []     i = j = 0     while i < len(left) and j < len(right):        if left[i].cgpa >=
right[j].cgpa:
          result.append(left[i])
i += 1        else:
          result.append(right[j])
j += 1
   result.extend(left[i:])
result.extend(right[j:])     return
result

def display_top_students(students: List[Student], top_n: int = 10):
   print(f"Top {top_n} Students by CGPA:")
for s in students[:top_n]:
       print(s)

# Generate random student data for performance test def
generate_students(n: int) -> List[Student]:
   return [Student(f"Student{i}", f"R{i:04d}", round(random.uniform(5.0, 10.0), 2)) for i in
range(n)]

students = generate_students(10000)
```

```
# Measure Quick Sort start = time.time() qs_sorted =
quick_sort_students(students.copy()) qs_time =
time.time() - start

# Measure Merge Sort start = time.time() ms_sorted =
merge_sort_students(students.copy()) ms_time =
time.time() - start

print(f"Quick Sort Time: {qs_time:.4f} seconds")
print(f"Merge Sort Time: {ms_time:.4f} seconds")
display_top_students(qs_sorted)

# ----------------------------------------
# Task 2: Bubble Sort with AI Explanations
# ----------------------------------------

def bubble_sort(arr: List[float]) -> List[float]:
    """
    Sorts a list using Bubble Sort algorithm.
    Time Complexity: O(n^2) in worst and average case.
    """     n =
len(arr)     for i in
range(n):
        # Each pass moves the largest unsorted element to its correct position
for j in range(0, n - i - 1):
```

```
        # Compare adjacent elements
if arr[j] > arr[j + 1]:
            # Swap if elements are in wrong order
arr[j], arr[j + 1] = arr[j + 1], arr[j]
      # If no swaps occurred, the list is sorted
return arr

# Bubble Sort Complexity:
# Best Case: O(n) (when already sorted)
# Average/Worst Case: O(n^2)

# ----------------------------------------------
# Task 3: Quick Sort & Merge Sort Comparison
# ----------------------------------------------

def quick_sort(arr: List[int]) -> List[int]:
   """
   Recursively sorts a list using Quick Sort.
   Average/Best: O(n log n), Worst: O(n^2)
   """    if len(arr) <= 1:        return arr    pivot = arr[0]
left = [x for x in arr[1:] if x < pivot]    right = [x for x in
arr[1:] if x >= pivot]    return quick_sort(left) + [pivot]
+ quick_sort(right)

def merge_sort(arr: List[int]) -> List[int]:
   """
   Recursively sorts a list using Merge Sort.
```

```
    Always O(n log n)
    """     if len(arr) <= 1:
return arr     mid = len(arr) // 2
left = merge_sort(arr[:mid])
right = merge_sort(arr[mid:])
return merge(left, right)

def merge(left: List[int], right: List[int]) -> List[int]:    result = []    i = j = 0    while i
< len(left) and j < len(right):        if left[i] < right[j]:
            result.append(left[i])
i += 1        else:
            result.append(right[j])
j += 1
    result.extend(left[i:])
result.extend(right[j:])     return
result

# Compare on random, sorted, reverse-sorted lists test_lists = {
    "random": [random.randint(0, 10000) for _ in range(1000)],
    "sorted": list(range(1000)),
    "reverse": list(range(999, -1, -1))
}

for name, lst in test_lists.items():
    t1 = time.time()     quick_sort(lst.copy())     t2 = time.time()
merge_sort(lst.copy())     t3 = time.time()     print(f"{name.capitalize()} List:
QuickSort {t2-t1:.5f}s, MergeSort {t3-t2:.5f}s") # Complexity Explanation:
```

```
# Quick Sort: Average/Best O(n log n), Worst O(n^2) (rare, e.g., sorted input with bad pivots)
# Merge Sort: Always O(n log n), but uses extra space

# ----------------------------------------------------
# Task 4: Inventory Management System (Search & Sort)
# ----------------------------------------------------

class Product:
    def __init__(self, pid: str, name: str, price: float, qty: int):
        self.pid = pid
self.name = name
self.price = price        self.qty
= qty

    def __repr__(self):
        return f"{self.pid}: {self.name} - ${self.price}, Qty: {self.qty}"

def binary_search_products(products: List[Product], key: str, by: str = "pid") -> Product:
    """Binary search for product by id or name (requires sorted list)."""
left, right = 0, len(products) - 1     while left <= right:
        mid = (left + right) // 2        val =
getattr(products[mid], by)        if val
== key:
            return products[mid]
elif val < key:
            left = mid + 1
else:
```

```
        right = mid - 1
return None

def sort_products(products: List[Product], by: str = "price") -> List[Product]:
    """Sort products by price or quantity using Merge Sort (efficient for large datasets)."""
if len(products) <= 1:        return products     mid = len(products) // 2     left =
sort_products(products[:mid], by)     right = sort_products(products[mid:], by)     return
merge_products(left, right, by)

def merge_products(left: List[Product], right: List[Product], by: str) -> List[Product]:
result = []     i = j = 0     while i < len(left) and j < len(right):        if getattr(left[i], by) <=
getattr(right[j], by):
        result.append(left[i])
i += 1        else:
        result.append(right[j])
j += 1
    result.extend(left[i:])
result.extend(right[j:])     return
result

# Recommended Algorithms Table
print("\nOperation\tAlgorithm\t\tJustification") print("Search by ID\tHash
Map\tO(1) lookup, best for large datasets") print("Search by Name\tHash
Map\tO(1) lookup, best for large datasets") print("Sort by Price\tMerge
Sort\tStable, O(n log n), good for large data") print("Sort by Qty\tMerge
Sort\tStable, O(n log n), good for large data")
```

```
# Example inventory inventory = [Product(f"P{i:04d}", f"Product{i}",
round(random.uniform(10, 500), 2), random.randint(1, 1000)) for i in range(1000)]
# Build hash maps for fast search inventory_by_id =
{p.pid: p for p in inventory} inventory_by_name =
{p.name: p for p in inventory}

# Example search and sort searched =
inventory_by_id.get("P0001") sorted_by_price =
sort_products(inventory, "price") sorted_by_qty =
sort_products(inventory, "qty")

# ----------------------------------------------------
# Task 5: Real-Time Stock Data Sorting & Searching
# ----------------------------------------------------

class Stock:     def __init__(self, symbol: str, open_price: float,
close_price: float):
        self.symbol = symbol
self.open_price = open_price
self.close_price = close_price    def
percent_change(self):        return
((self.close_price - self.open_price) /
self.open_price) * 100

    def __repr__(self):
        return f"{self.symbol}: {self.open_price} -> {self.close_price}
({self.percent_change():.2f}%)"
```

```
def heapify(arr: List[Stock], n: int, i: int):
   # Heapify subtree rooted at i for descending order by percent change
largest = i     l = 2 * i + 1     r = 2 * i + 2     if l < n and arr[l].percent_change()
> arr[largest].percent_change():        largest = l     if r < n and
arr[r].percent_change() > arr[largest].percent_change():        largest = r
if largest != i:
       arr[i], arr[largest] = arr[largest], arr[i]
heapify(arr, n, largest)

def heap_sort_stocks(arr: List[Stock]):
n = len(arr)     for i in range(n // 2 - 1, -1,
-1):
       heapify(arr, n, i)     for i in
range(n - 1, 0, -1):        arr[0],
arr[i] = arr[i], arr[0]
heapify(arr, i, 0)
arr.reverse()  # For descending
order

# Simulate stock data stocks = [Stock(f"STK{i:03d}",
round(random.uniform(100, 1000), 2), round(random.uniform(100,
1000), 2)) for i in range(1000)]
# Build hash map for fast symbol search stock_map =
{s.symbol: s for s in stocks}

# Heap Sort stocks_copy =
stocks.copy() start = time.time()
```

```
heap_sort_stocks(stocks_copy)

heap_time = time.time() - start

# Standard sorted() start = time.time() sorted_stocks = sorted(stocks, key=lambda s:

s.percent_change(), reverse=True) sorted_time = time.time() - start

print(f"\nHeap Sort Time: {heap_time:.5f}s, sorted() Time: {sorted_time:.5f}s")

print("Top 5 Stocks by % Change (Heap Sort):") for s in stocks_copy[:5]:

    print(s)

# Hash Map Search symbol =

"STK005" start = time.time()

result = stock_map.get(symbol)

hash_time = time.time() - start

# Linear Search start = time.time() result2 = next((s for s in

stocks if s.symbol == symbol), None) linear_time = time.time() -

start

print(f"\nHash Map Search Time: {hash_time:.8f}s, Linear Search Time:
{linear_time:.8f}s") print(f"Stock

{symbol}: {result}")

# Trade-off Analysis:

# - Heap Sort: O(n log n), good for large data, in-place, not stable.

# - sorted(): Highly optimized, stable, uses Timsort (O(n log n)).

# - Hash Map: O(1) average lookup, best for frequent searches.
```

# - Linear Search: O(n), slow for large data.

OUTPUT:

```
PS C:\Users\HARSHAVARDHAN\OneDrive\Desktop\AI-CODING> & C:/Users/HARSHAVARDHAN/
e/Desktop/AI-CODING/ai-coding-4.5.py
Enter social media post: 1
Category: Inquiry
PS C:\Users\HARSHAVARDHAN\OneDrive\Desktop\AI-CODING>
```